# Cloud Computing
# LAB-4

Name: Vardha Kathura
SRN: PES1UG23AM343
Section: F

In this step, Docker installation was verified using the docker run hello-world command.
The successful execution and output confirm that the Docker client is able to communicate with the Docker daemon and pull images from Docker Hub, indicating that Docker is correctly installed and working.

```
'~$ternship_Report_Zimplistic_Technical.docx'
PES1UG23AM343:~/Downloads$ cd Docker_lab_advanced
PES1UG23AM343:~/Downloads/Docker_lab_advanced$ pwd
/home/vardha/Downloads/Docker_lab_advanced
PES1UG23AM343:~/Downloads/Docker_lab_advanced$ ls
Dockerfile  chat.py  docker-compose.yml
PES1UG23AM343:~/Downloads/Docker_lab_advanced$ export SRN=PES1UG23AM343
PES1UG23AM343:~/Downloads/Docker_lab_advanced$ echo $SRN
PES1UG23AM343
PES1UG23AM343:~/Downloads/Docker_lab_advanced$ export PS1="[\u@$SRN \w]\$ "
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ docker run hello-wor
ld

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent
it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$
```

In this step, Docker Compose was verified using the docker compose version command.
The output confirms that Docker Compose is properly installed and available for orchestrating multiple container services using a YAML configuration file.

```
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ docker compose versi
on
Docker Compose version v5.0.1
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$
```

In this step, a Docker image for the chat application was built using the provided Dockerfile with the command docker build -t chat-app ..
The successful build confirms that all dependencies were installed correctly and the application is packaged into a reusable Docker image.

```
Run 'docker buildx build --help' for more information
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ docker build -t chat
-app .
[+] Building 38.5s (10/10) FINISHED                          docker:default
 => [internal] load build definition from Dockerfile                      0.1s
 => => transferring dockerfile: 323B                                      0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim        6.6s
 => [internal] load .dockerignore                                         0.1s
 => => transferring context: 2B                                           0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16     21.5s
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b     0.0s
 => => sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfe 1.29MB / 1.29MB     1.9s
 => => sha256:38513bd7256313495cdd83b3b0915a633cfa 29.78MB / 29.78MB     17.3s
 => => sha256:ea56f685404adf81680322f152d2cfec62115b30dda 251B / 251B     2.0s
 => => sha256:fc74430849022d13b0d44b8969a953f842f5 13.88MB / 13.88MB     13.8s
 => => extracting sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a0     2.3s
 => => extracting sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9     0.2s
 => => extracting sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a     1.3s
 => => extracting sha256:ea56f685404adf81680322f152d2cfec62115b30dda4     0.0s
 => [internal] load build context                                         0.1s
 => => transferring context: 4.17kB                                       0.0s
 => [2/5] RUN pip install pika                                            7.1s
 => [3/5] WORKDIR /app                                                    0.1s
 => [4/5] COPY chat.py .                                                  0.1s
 => [5/5] RUN mkdir -p /app/data                                          0.5s
 => exporting to image                                                    2.0s
 => => exporting layers                                                   1.2s
 => => exporting manifest sha256:277ca109413521362ce0c0806d379398c0ca     0.0s
 => => exporting config sha256:a47afdaf5214bb4eb80ebcef1606920e515ac6     0.0s
 => => exporting attestation manifest sha256:853ebe079ba9a7af29457287     0.1s
 => => exporting manifest list sha256:efcb5efb5392d688eb6e6a2583f62fe     0.0s
 => => naming to docker.io/library/chat-app:latest                        0.0s
```

In this step, the RabbitMQ message broker was started as a Docker container using the rabbitmq:3-management image.
The successful image pull and container startup confirm that RabbitMQ is running and ready to handle message queuing between different services.

```
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ docker run -d --name
 rabbitmq rabbitmq:3-management
Unable to find image 'rabbitmq:3-management' locally
3-management: Pulling from library/rabbitmq
d686b50ce172: Pull complete
beb0d870d438: Pull complete
d63c4408fdb7: Pull complete
7084ac8f4ab6: Pull complete
20043066d3d5: Pull complete
f4e7f62f4b80: Pull complete
3fadc669ef11: Pull complete
efa05c660812: Pull complete
5e1c03b67a78: Pull complete
14e6209d5fa8: Pull complete
7ec4c696b8bb: Download complete
16ef6b12768c: Download complete
Digest: sha256:e582c0bc7766f3342496d8485efb5a1df782b5ce3886ad017e2eaae442311
f69
Status: Downloaded newer image for rabbitmq:3-management
0ec1ffeee8205d8961b45be5fbc4443a3a5c19524a89993a0af8b8f6ef82c31a
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ |
```

In this step, the running status of the RabbitMQ container was verified using the docker ps command. The output confirms that the RabbitMQ container is active and running, exposing the required ports for message broker communication.

```
0ec1ffeee8205d8961b45be5fbc4443a3a5c19524a89993a0af8b8f6ef82c31a
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ docker ps
CONTAINER ID   IMAGE                   COMMAND                  CREATED
     STATUS          PORTS
                NAMES
0ec1ffeee820   rabbitmq:3-management   "docker-entrypoint.s…"   22 seconds a
go   Up 21 seconds   4369/tcp, 5671-5672/tcp, 15671-15672/tcp, 15691-15692/t
cp, 25672/tcp   rabbitmq
```

In this step, the chat application was run without attaching it to a Docker network.
The application failed to connect to RabbitMQ and fell back to standalone mode, demonstrating that containers are isolated by default and cannot communicate without an explicitly defined network.

```
cp, 25672/tcp   rabbitmq
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ docker run -it --nam
e user-a chat-app

--- No Previous Chat History ---

--- Chat Started (unknown-user connecting to localhost) ---
RabbitMQ not ready, retrying... (1/5)
RabbitMQ not ready, retrying... (1/5)
RabbitMQ not ready, retrying... (2/5)
RabbitMQ not ready, retrying... (2/5)
RabbitMQ not ready, retrying... (3/5)
RabbitMQ not ready, retrying... (3/5)
RabbitMQ not ready, retrying... (4/5)
RabbitMQ not ready, retrying... (4/5)
RabbitMQ not ready, retrying... (5/5)
RabbitMQ not ready, retrying... (5/5)

ERROR: Could not connect to RabbitMQ at localhost

ERROR: Could not connect to RabbitMQ at localhost

Falling back to standalone mode...
==================================================
STANDALONE MODE (No RabbitMQ)
==================================================

--- No Previous Chat History ---

Messages will not be sent to any other container.

Type message (standalone): Hello
Sent by unknown-user (standalone): Hello
Type message (standalone): What are you doing?
Sent by unknown-user (standalone): What are you doing?
Type message (standalone): ^C
Exiting standalone mode...
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$
```

In this step, the standalone chat container (`user-a`) was stopped and removed using the `docker rm -f` command.
This cleanup step ensures that no conflicting containers are running before proceeding with Docker network creation and multi-container communication.

```
Exiting standalone mode...
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ docker rm -f user-a
user-a
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$
```

In this step, a user-defined Docker network (chat-net) was created to enable container-to-container communication.
RabbitMQ was then restarted and attached to this network, allowing other containers on the same network to resolve and communicate with it using the container name.

```
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ docker rm -f user-a
user-a
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ docker network creat
e chat-net
897695ff2d1060a15e0ae4844120919ae8e0860c4ef5c080534ac95f869e208f
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ docker stop rabbitmq
rabbitmq
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ docker rm rabbitmq
rabbitmq
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ docker run -d --name
 rabbitmq --network chat-net rabbitmq:3-management
29059c5fe94d3b7e5a9290bf93ce7d19bcd4af1d23d1dcb86a4b60563f872cd1
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED
    STATUS         PORTS
             NAMES
29059c5fe94d   rabbitmq:3-management   "docker-entrypoint.s…"   5 seconds ag
o   Up 4 seconds   4369/tcp, 5671-5672/tcp, 15671-15672/tcp, 15691-15692/tcp
, 25672/tcp   rabbitmq
[vardha@PES1UG23AM343 ~/Downloads/Docker_lab_advanced]$
```

In this step, two chat application containers (user-a and user-b) were started on the same Docker network and connected to RabbitMQ.
Messages sent from one container were successfully received by the other, demonstrating container-to-container communication using a message broker over a user-defined network.

```
C:\Windows\System32\wsl.ex
 -e RABBIT_HOST=rabbitmq \
 -e QUEUE_NAME=queue_a \
 -e TARGET_QUEUE=queue_b \
 chat-app

--- No Previous Chat History ---

--- Chat Started (unknown-user connecting to rabbitmq) ---
Type message: hello
Sent by unknown-user: hello
Type message:
Received from unknown-user: what are you doing?
Type message: nothing much
Sent by unknown-user: nothing much
Type message:
Received from unknown-user: all good?
Type message: yes
Sent by unknown-user: yes
Type message:
```

```
C:\Windows\System32\wsl.ex
 -e RABBIT_HOST=rabbitmq \
 -e QUEUE_NAME=queue_b \
 -e TARGET_QUEUE=queue_a \
 chat-app

--- No Previous Chat History ---

--- Chat Started (unknown-user connecting to rabbitmq) ---
Type message:
Received from unknown-user: hello
Type message: what are you doing?
Sent by unknown-user: what are you doing?
Type message:
Received from unknown-user: nothing much
Type message: all good?
Sent by unknown-user: all good?
Type message:
Received from unknown-user: yes
Type message:
```

In this step, the docker-compose.yml file was examined to define multiple services, networks, and environment variables.
The YAML configuration specifies RabbitMQ and two chat users as services, enabling automated setup, networking, and dependency management using Docker Compose.

```yaml
version: "3.8"

services:
  rabbitmq:
    image: rabbitmq:3-management
    networks:
      - chat-net

  user-a:
    build: .
    stdin_open: true
    tty: true
    environment:
      USER_NAME: user-a
      RABBIT_HOST: rabbitmq
      QUEUE_NAME: queue_a
      TARGET_QUEUE: queue_b
    # STAGE 5: Uncomment this for persistence demonstration
    #volumes:
      #- chat-history:/app/data
    networks:
      - chat-net
    depends_on:
      - rabbitmq

  user-b:
    build: .
    stdin_open: true
    tty: true
    environment:
      USER_NAME: user-b
      RABBIT_HOST: rabbitmq
```